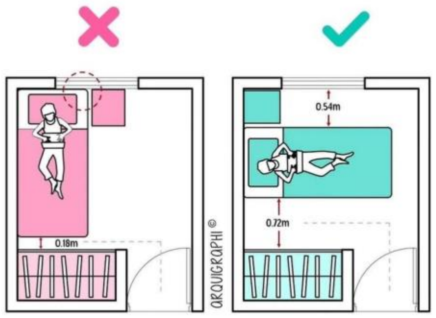
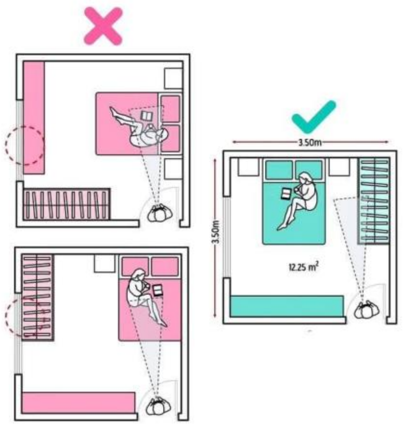
RL Description
Problem

Furnish a Compact Bedroom

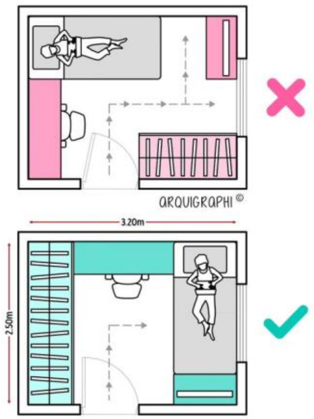
what to pick . where to place . which way to face
A common, everyday problem — but with no clear optimal solution.



1.Function



2.Comfort



3.Efficiency

We formulate this combinatorial, multi-objective design task as an MDP
and explore how an RL agent learns to navigate the trade-offs.

Images: ARQUIGRAPHI © (@arquigraphi)

RL Description

Game & Abstraction

Before formulation, we turn this open-ended design task into a tractable RL game with
- **a fixed grid, a finite catalog, and sequential placement**.

Interactive HTML preview:
https://huch64.github.io/CA6126_myLittleBedroom/my_little_bedroom.html
we built this to verify the env before training.

The RL Game

① pick a furniture piece

② place at (x, y, orientation) — bed-first

③ repeat (max 6 pieces / 8 steps)

④ issue DONE; final layout is scored

- Catalog: 18 items / 5 categories
- Limits: ≤1 per category (≤2 NS), bed-first, max 8 steps
- Each episode randomizes room / door / window — forcing generalization, not memorization.

RL Formulation

MDP - State

The state captures the room layout and all furniture placed so far.

S = (grid, placements)

grid: 26×22 cells (0.15m/cell)

- room boundary (randomized)
- door: 6-cell opening randomly positioned along the bottom wall
- Window: opening on one of {top, left, right} walls

placements: the set of pieces dropped so far:

- each = (item, position, orientation)

$|S| \approx 10^{11} \sim 10^{12}$

(combinatorial in furniture × room/door/window)

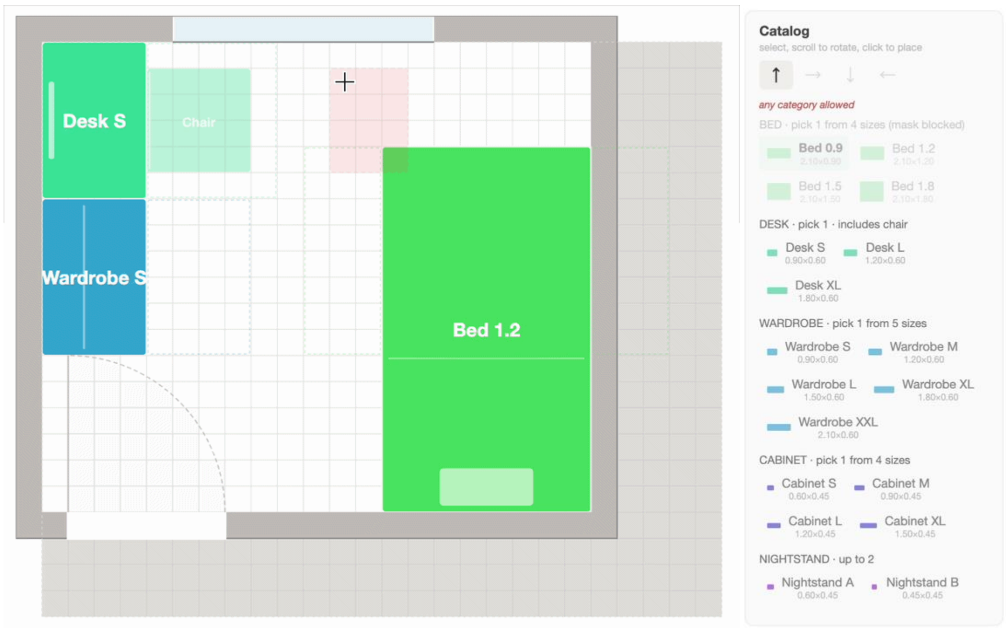
2,640 room configs × ~10^8 valid layouts per room.
Constraints (no overlap, category caps, bed-first) reduce a naive 10^{25} by ~10^{13}×.
Still far beyond tabular — requires function approximation.

$\mu(s_0)$: random room config, empty furniture set

RL Formulation

MDP – Action & Transition

The agent decides what to place, where, and how — one piece at a time, no take-backs.



furID [x , y] Ori

A = Piece × cell × orientation ∪ {DONE}

18 26×22 4 1

|A| = 18 · (26·22) · 4 + 1 = 41,185

A(s) ⊆ A

feasible set shrinks as room fills
(~10^3 early → single digits near end)
enforced via action masking (MaskablePPO)

T: P(s’|s,a) deterministic

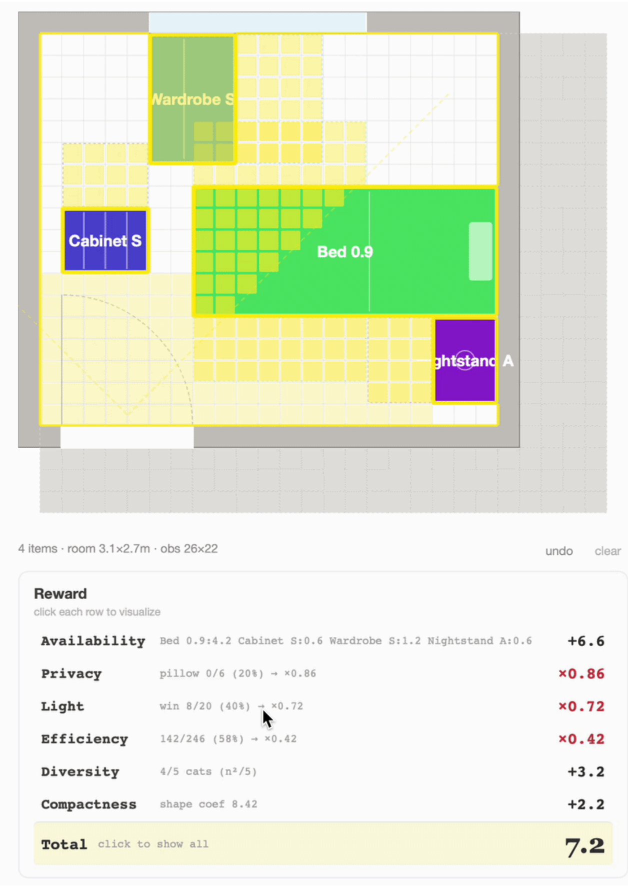
the placement writes its footprint into the grid.

Termination: a = DONE or step = 8

RL Formulation

MDP - Reward

The reward scores of six component computed on the final layout.



$$\underbrace{R(s\_T) = A}_{\text{Base}} \cdot \underbrace{p \cdot l \cdot e}_{\text{Quality}} + \underbrace{div + comp}_{\text{Bonus}}$$

at terminal state

R(s, a, s′) = 0

∀ non-terminal s′

R(s_T) = 0

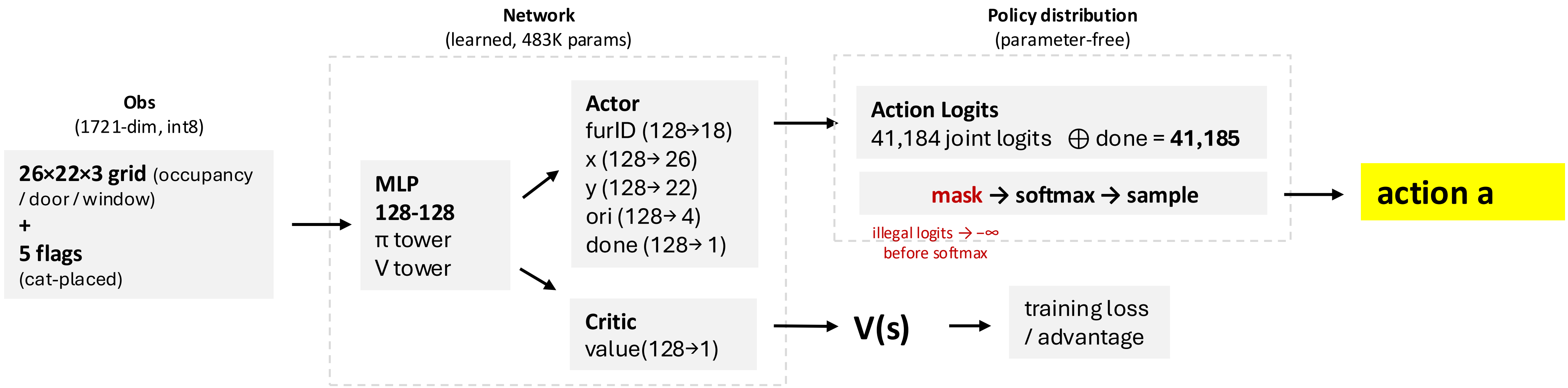
if no bed in s_T

Role	Component	Formula	Range	Measures
base	**A**vailability	Σ area × 0.05	0 – 16.3	usable furniture placed
Quality × on A	**p**rivacy	1 − 0.7 · exposure	[0.3, 1]	bed hidden from door
	light	1 − 0.7 · win_ratio	[0.3, 1]	window unblocked
	efficiency	1 − waste_ratio	[0, 1]	floor reachable
Bonus + on R	**div**ersity	$n\_cat^2 / 5$	0.2 – 5.0	category variety
	compactness	$5(1-(p/\sqrt{a}-4)/8)$	[0, 5]	free space regularity

RL Solution

Network Architecture

The agent observes the room grid and outputs a placement decision through a factored policy network — masked to only legal moves.



Trained with MaskablePPO (sb3-contrib)

Action masking

hard constraints in mask, soft preferences in reward — **keeps the learning signal clean**.

Example: if bed is already placed, all bed actions are masked → agent can't place a second bed.

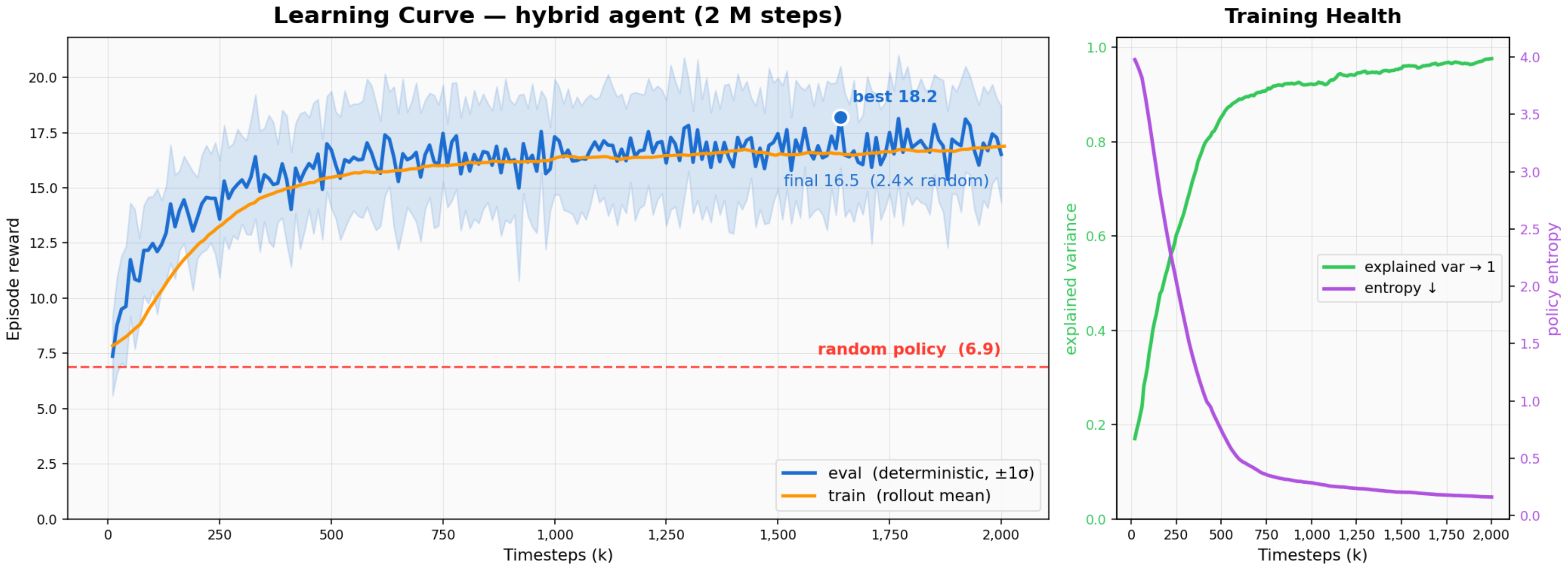
RL Solution

Training Curve

Trained with default PPO hyperparameters. Most gains in first 750K steps — then converges.

Reward: 6.9 (random) → 16.5 (2.4×).

Train ≈ eval — generalizing across random rooms, not memorizing.



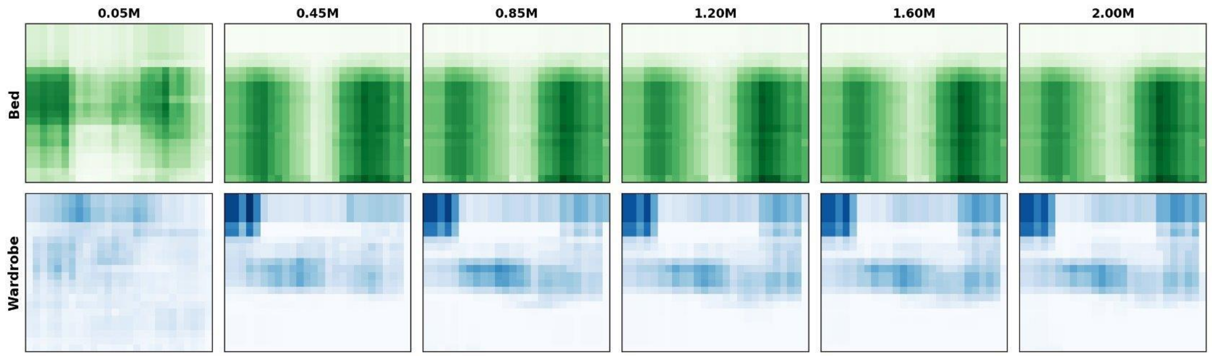
PPO hyperparams	lr=3e-4→0 γ=0.99 λ=0.95 clip=0.2 epochs=10 ent=0.01 vf=0.5 grad_norm=0.5
Infrastructure	Algorithm=MaskablePPO envs=20 batch=128 n_steps=512
Outcome	Total steps=2M Wall-clock=~78min

RL Solution
Emergent & Robustness

How does the agent's spatial behavior develop, and how robust is it across room sizes?

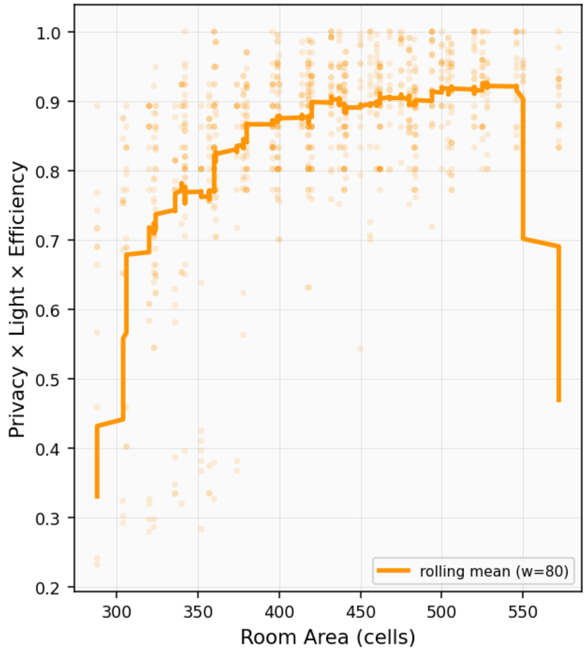
Evaluated on 1000 episodes with randomized room configurations (288–572 cells).

Spatial Maturation over Training



Spatial strategy matures gradually over training: from random placement to structured corner-and-wall preference.

Quality vs Room Size



Quality peaks in mid-sized rooms (~400-500 cells).
Too small — furniture can't fit well.
Too large — wasted space drags down efficiency.

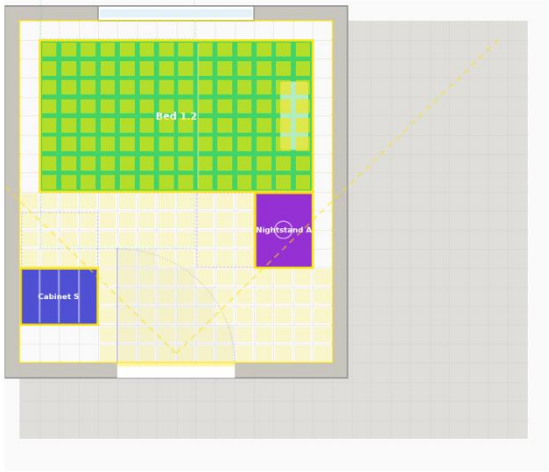
RL Solution

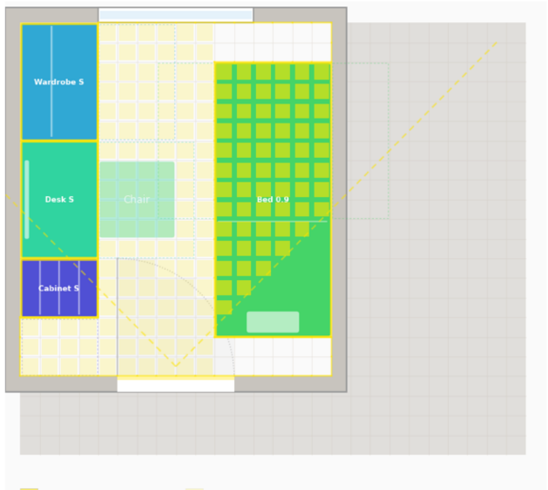
Random VS Trained

Random model

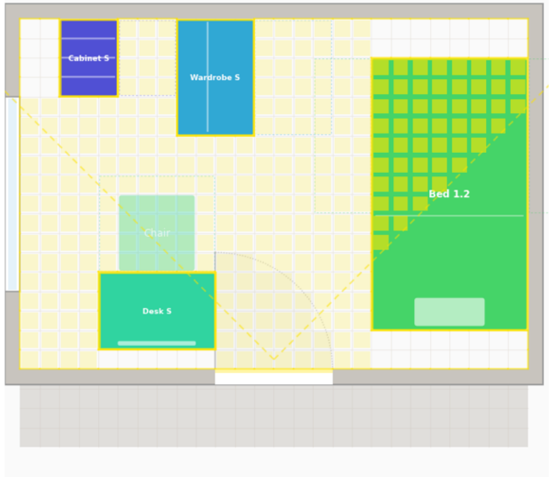
Trained model

Room	16 × 18 cells
Door	bottom wall, x=5, w=6
Window	top wall, x=4, w=8
Swing	28 cells reserved





Room	18 × 22 cells
Door	bottom wall, x=6, w=6
Window	right wall, x=5, w=12
Swing	28 cells reserved

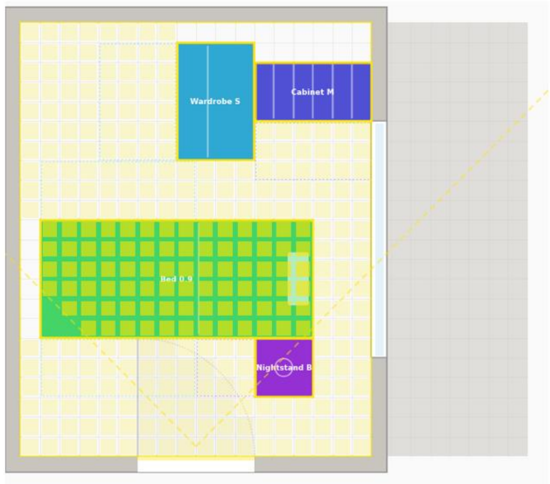


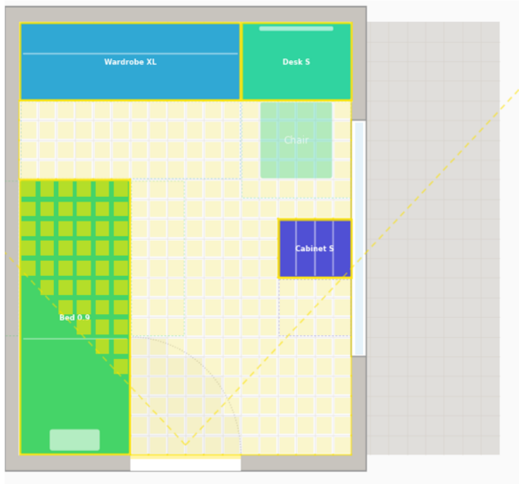
RL Solution
Random VS Trained

Random model

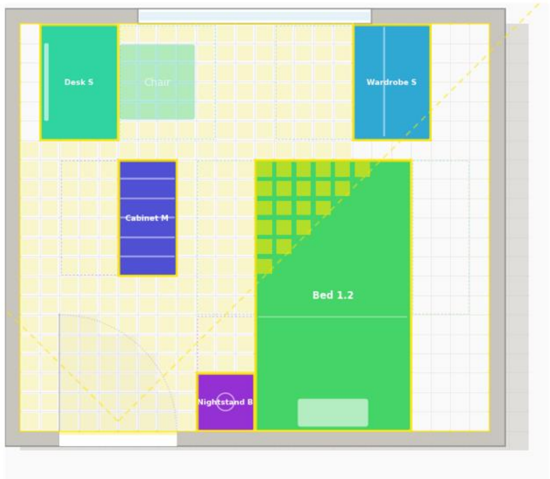
Trained model

Room	26 × 18 cells
Door	bottom wall, x=10, w=6
Window	left wall, x=4, w=10
Swing	28 cells reserved





Room	24 × 21 cells
Door	bottom wall, x=2, w=6
Window	top wall, x=6, w=12
Swing	28 cells reserved

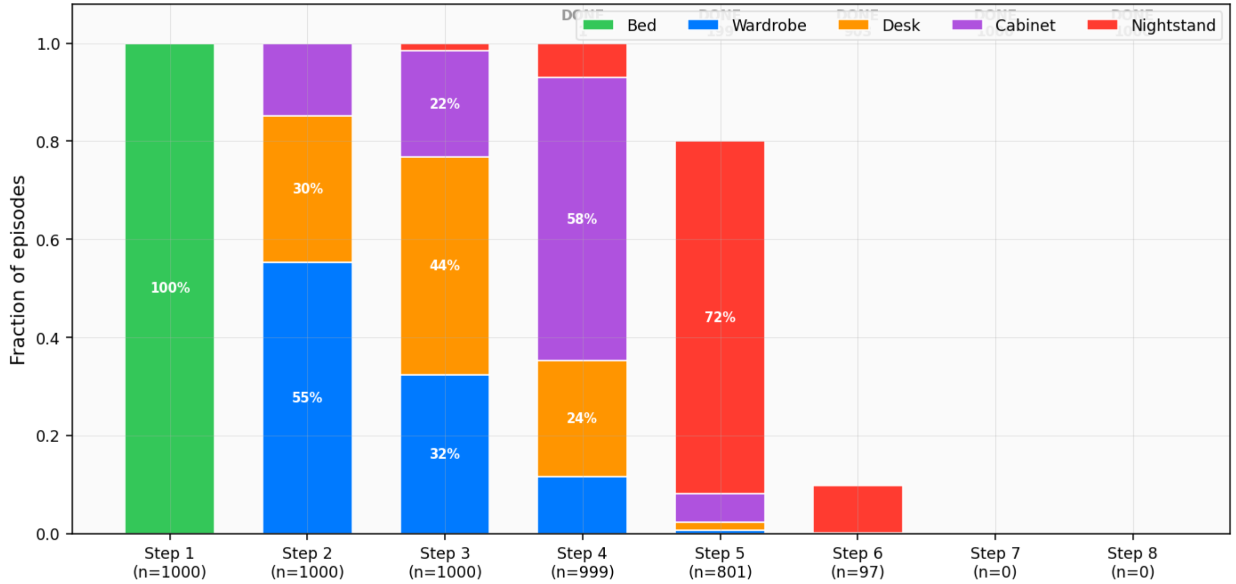


RL Solution

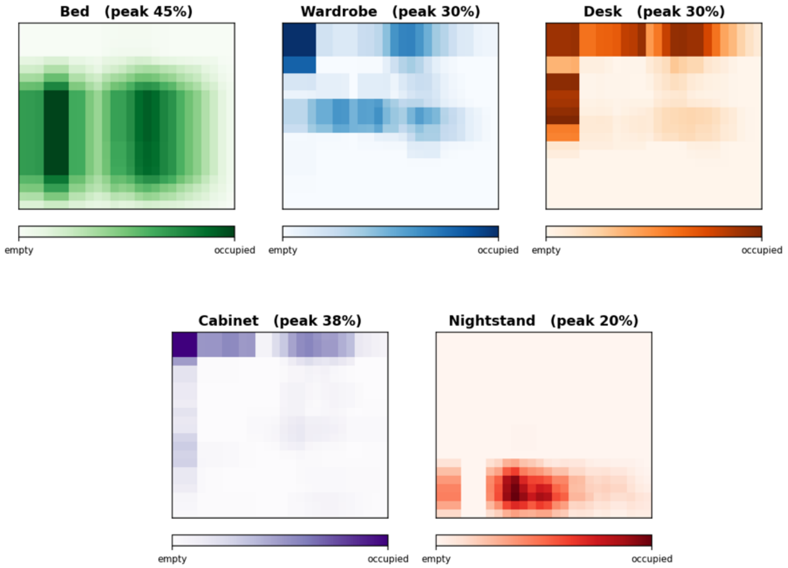
Trained Agent Preference

Spatial preferences ***<u>emerge</u>*** from reward:

Placement Order [when]



Spatial Preference [where]



-large pieces first
-small pieces last
-done in 5 steps.

-bed retreats from the door
-wardrobe hugs walls
-small pieces fill remaining gaps

Reward Composition

"+" and "×"

During reward shaping, we noticed that ***addition*** and ***multiplication*** carry fundamentally different design implications:

× acts as a **gate** —
if any factor is poor, the whole score drops.
It enforces minimum standards.

+ acts as **negotiation**—
a weak dimension can be compensated by a strong one. Everything is tradeable.

This led us to test three composition styles using the same 6 components:

Hybrid (default)
$R = A \cdot p \cdot l \cdot e + div + comp$
gates on quality, bonuses add freely

Additive
$R = A + 5p + 5l + 5e + div + comp$
all objectives negotiate equally

Multiplicative
$R = A \cdot p \cdot l \cdot e \cdot div \cdot comp$
any zero kills everything

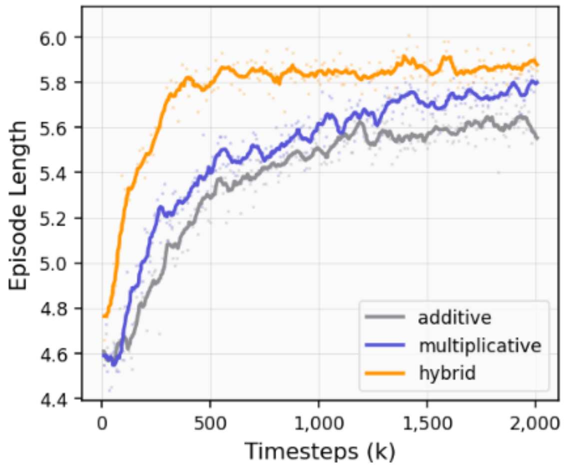
A = availability (usable furniture area)
p = privacy (bed hidden from door)
l = light (window unblocked)
e = efficiency (floor reachable)
div = diversity (category variety)
comp = compactness (free space regularity)

What happens when we train the same agent under each composition?

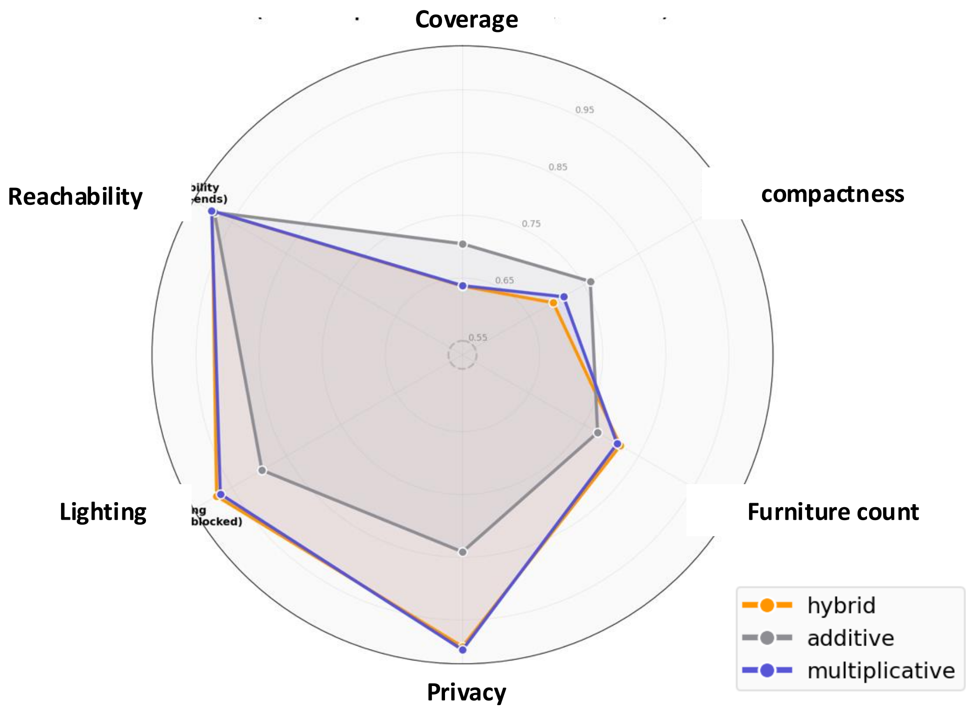
Reward Composition

Training Dynamics & Behavioral Profiles

Episode Lenth



Agent Behavior Profile



Deeper Planning Horizon:

Hybrid (orange) sustains the highest average steps (~5.9) ,which means putting more furniture into the bedroom, proving superior multi-step placement over **Additive** and **Multiplicative** (early termination).

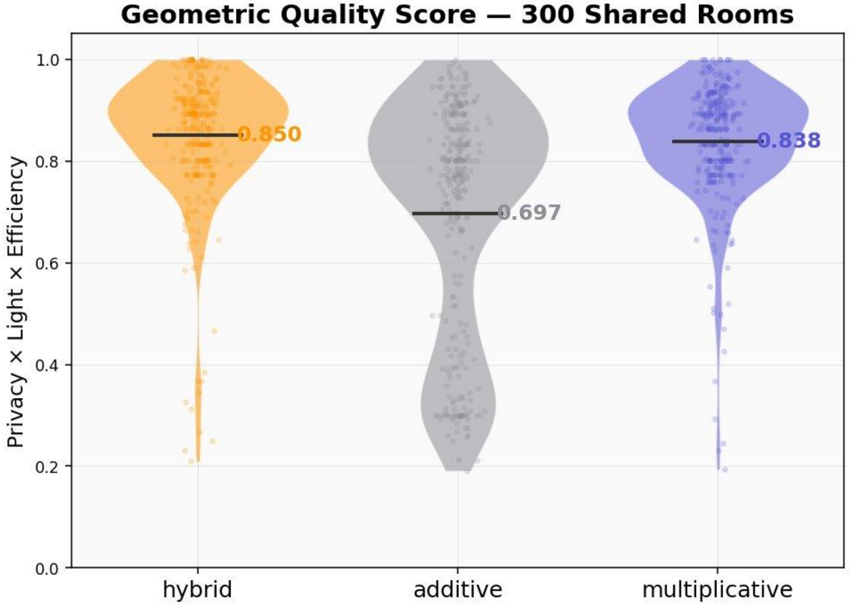
Strategic Divergence Explored :

Additive (Grey)**:** Blindly maximizes Coverage (greedy footprint) and Compactness at the catastrophic expense of breaching critical Lighting and Privacy redlines.

Multiplicative & **Hybrid**: Enforce non-negotiable physical constraints, self-organizing a well-balanced, human-aligned behavioral profile.

Reward Composition
Additive: Reward Hacking

Layout Quality Comparison



- **The Additive Illusion:** Achieves high coverage and compactness but blindly maximizes furniture area.

- **Physical Collapse:** Fails completely under strict P*L*E metrics, generating uninhabitable dead-ends.

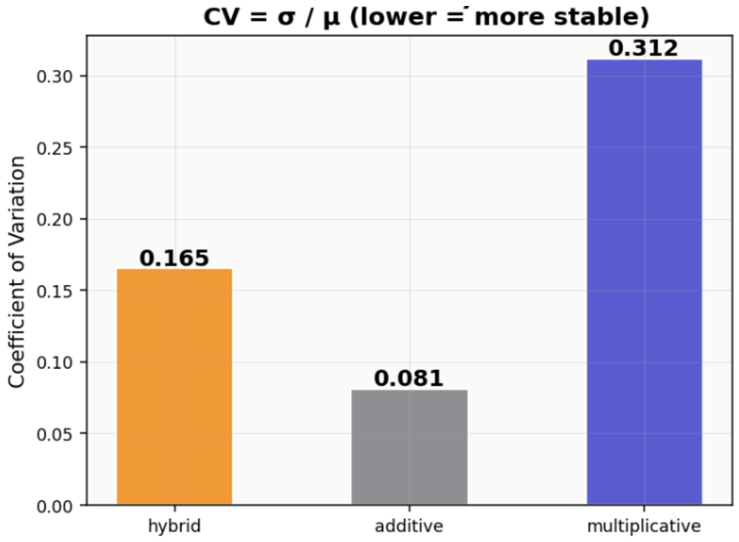
Case Study (Additive):

Completely ignoring privacy constraints and partially blocks natural light to earn raw area points.

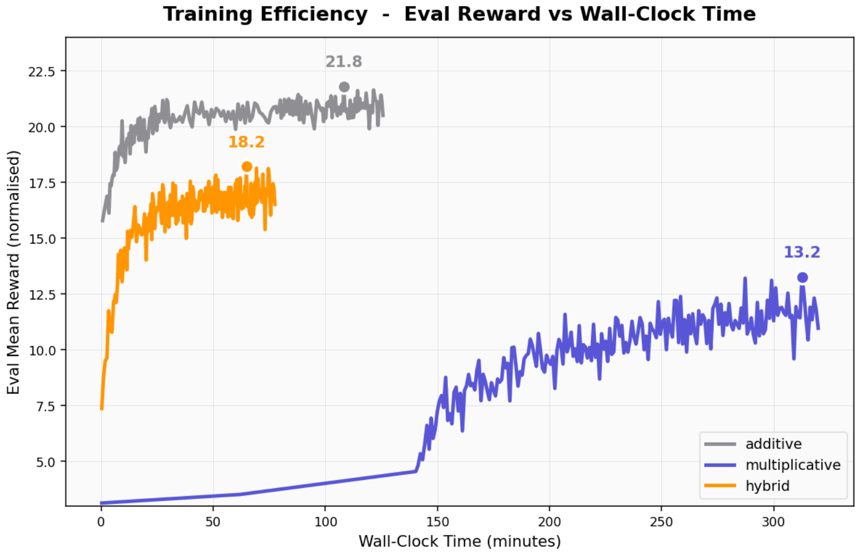
Reward Composition

Multiplicative: Gradient Starvation

How stable is each agent?



How fast does each agent learn?



The "Fake Stability" Illusion:
Additive's low CV reflects rigid, mechanical cheating. Hybrid's healthy variance (0.165) proves dynamic, adaptive generalization.

Multiplicities' Collapse:
Strict penalty causes severe gradient starvation, stalling training for the first 140 minutes.
At the same time, Hybrid is **4.2x Speedup** and Converges efficiently in just **76 minutes**.

Reward Composition
as a Design Language

The same design intent, expressed through different reward compositions, leads to qualitatively different agent behaviors.

Hybrid (default)

R = A · p · l · e + div + comp

Quality must pass, bonuses are extra.

→ Balanced: high quality, most furniture placed.

Additive

R = A + 5p + 5l + 5e + div + comp

Bad dimensions can be hidden by good ones.

→ High score, but cheats — ignores privacy and light.

Multiplicative

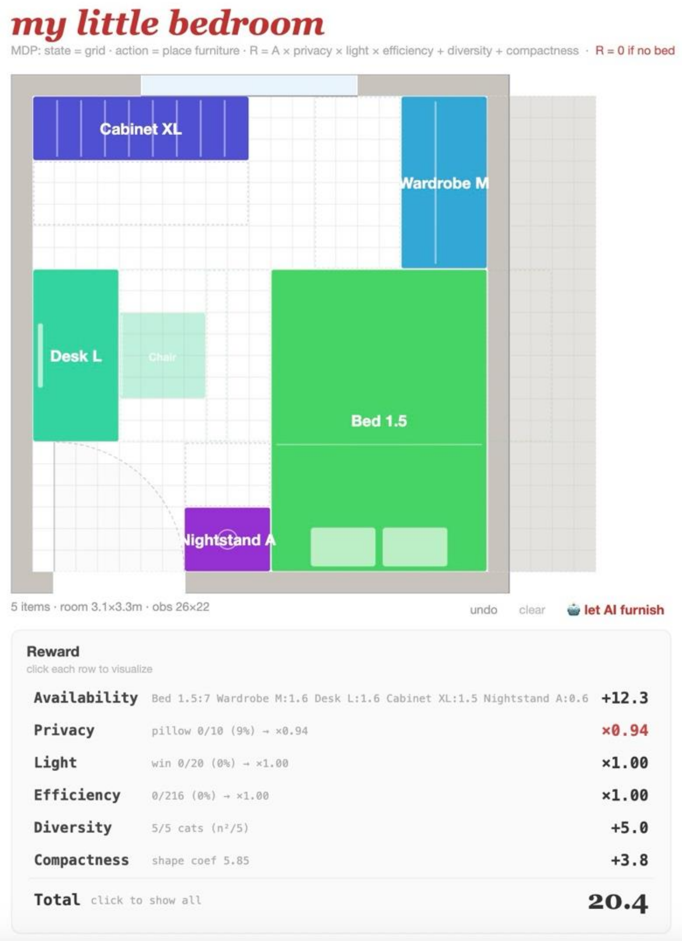
R = A · p · l · e · div · comp

One bad dimension ruins everything.

→ Too strict — agent can't learn from early mistakes.

We observe that ***multiplicative*** terms enforce quality floors, while ***additive*** terms express soft preferences — ***choosing between them*** is a design decision, not just a technical one.

Conclusion



Trained Agent — Hybrid
(best checkpoint)

What we did: *RL learns Spatial Layout*

Bedroom furnishing as MDP. MaskablePPO, 2.4 × random baseline. Spatial preferences emerge from reward — not programmed.

What we found: *Reward Composition as a Design Language*

composition matters: × enforces floors, + expresses preferences. Same components, different composition → different behavior.

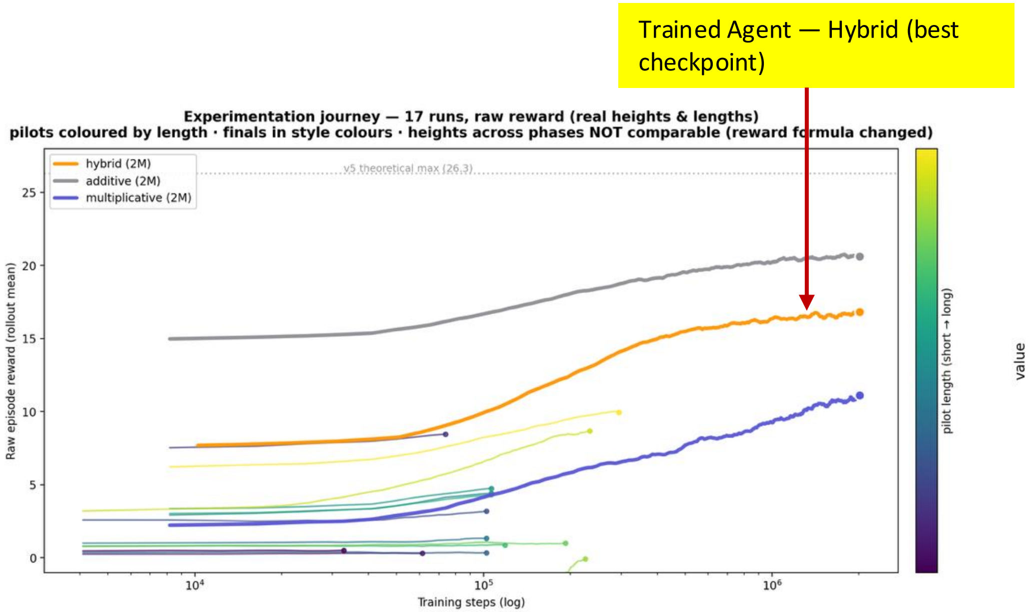
Limitations: *Grid is Flat, Constants are Manual*

Hand-tuned constants. Flat grid — spatial relations only implicit.

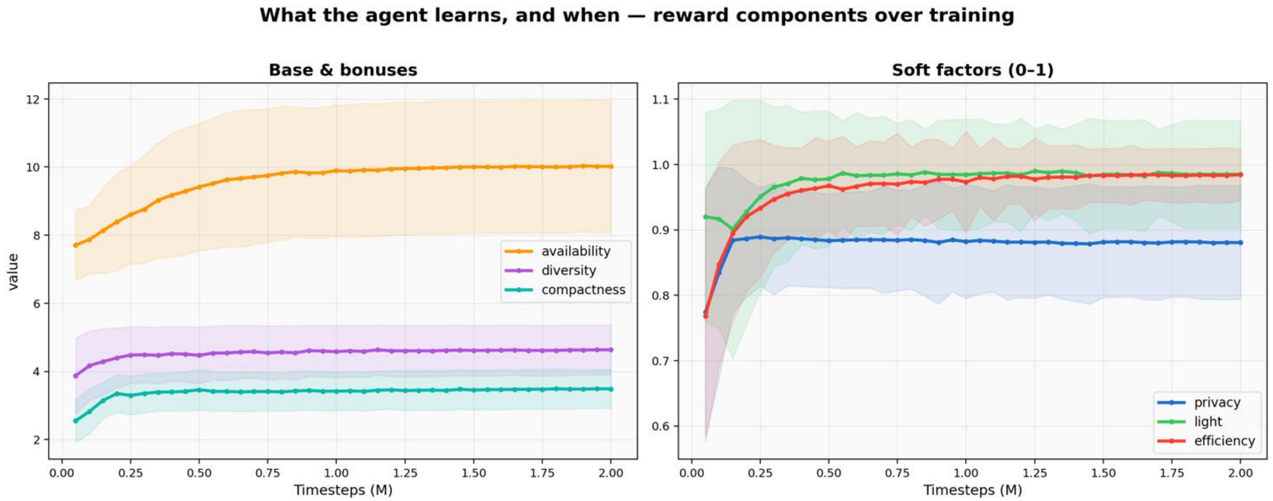
Future work: *Graph Structure (cf. House-GAN++)*

Graph-based representation (cf. House-GAN++, Nauata et al. 2021) could make relations explicit — adjacency, facing, blocking as edges. Bridge reward-driven RL with relation-driven generation.

Appendix
Training Journey



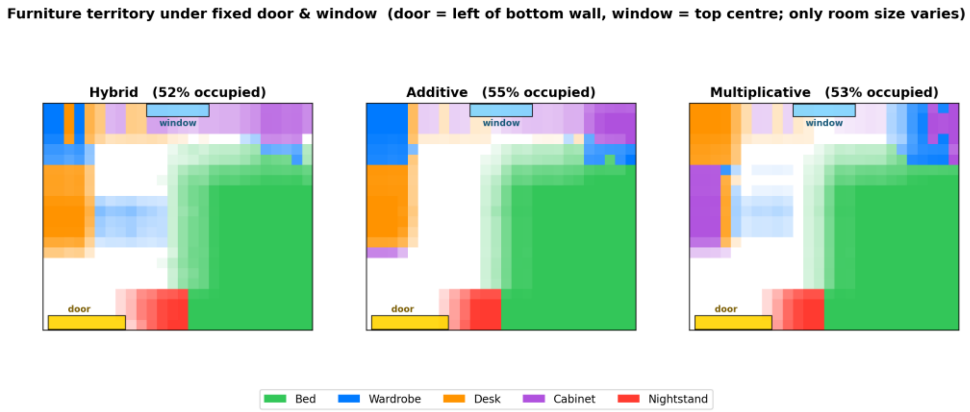
17 experimental runs across reward iterations.



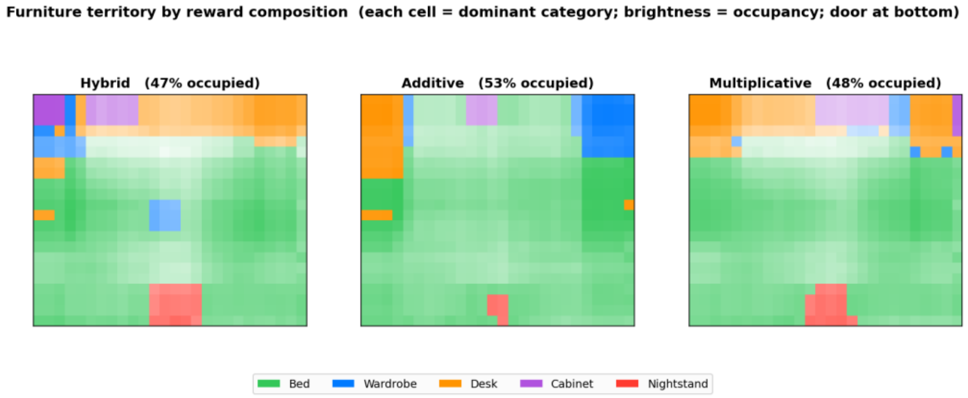
Reward component breakdown shows availability and diversity drive most gains; soft factors stabilize early.

Appendix

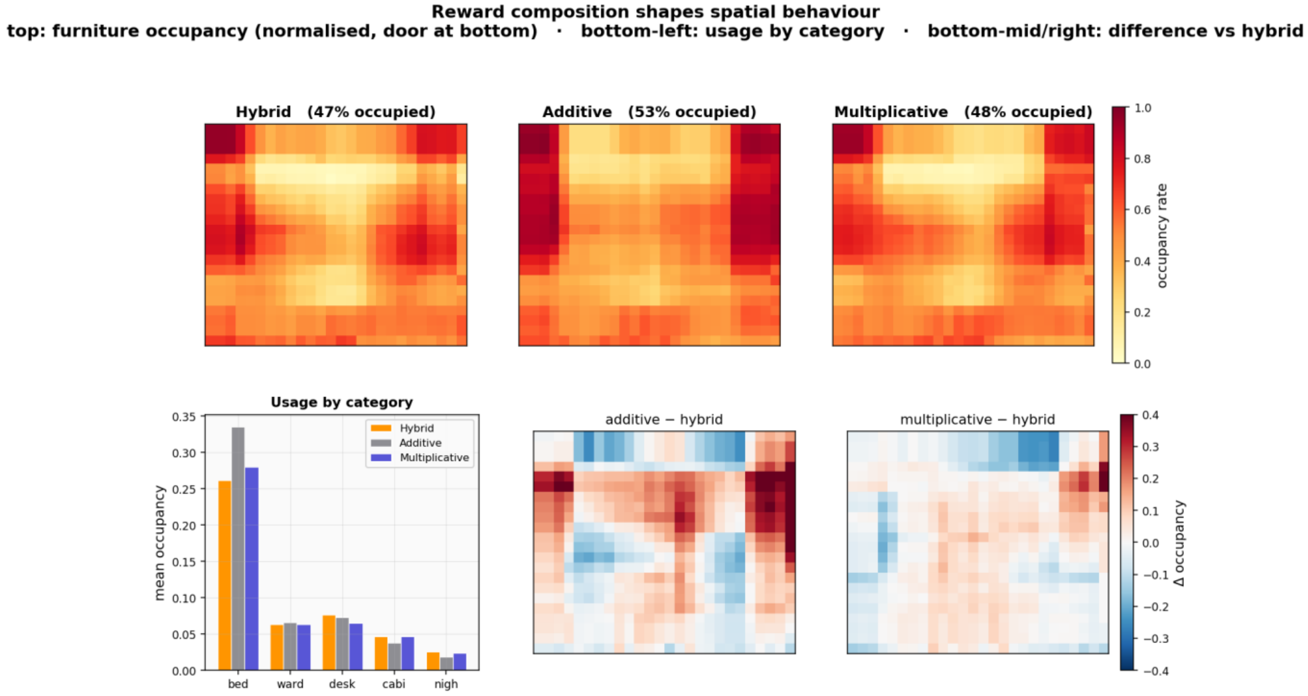
Reward Combination Heatmap



Fixed door/window, showing furniture placement patterns.



Spatial territory comparison across three compositions.



Normalized occupancy and category usage differences.